

```
terminal
11635 INFO: Including run-time hook /home/codio/anaconda3/envs/day/lib/python3.11/site-packages/PyInstaller/hooks/rthooks/pyl_rth_pkgutil.py'
11639 INFO: Including run-time hook /home/codio/anaconda3/envs/day/lib/python3.11/site-packages/PyInstaller/hooks/rthooks/pyl_rth_multiprocessing.py'
11651 INFO: Looking for dynamic libraries
14414 INFO: Looking for eggs
14415 INFO: Python library not among binary dependencies. Performing additional search...
14455 INFO: Using Python library /home/codio/anaconda3/envs/day/lib/libpython3.11.so.1.0
14458 INFO: Warnings written to /home/codio/workspace/day/build/day/warn-day.txt
14500 INFO: Graph cross-reference written to /home/codio/workspace/day/build/day/xref-day.html
14517 INFO: checking PYZ
14518 INFO: Building PYZ because PYZ-00.toc is non existent
14518 INFO: Building PYZ (ZlibArchive) /home/codio/workspace/day/build/day/PYZ-00.pyz
15166 INFO: Building PYZ (ZlibArchive) /home/codio/workspace/day/build/day/PYZ-00.pyz completed successfully.
15175 INFO: checking PKG
15175 INFO: Building PKG because PKG-00.toc is non existent
15175 INFO: Building PKG (CArchive) day.pkg
28993 INFO: Building PKG (CArchive) day.pkg completed successfully.
28996 INFO: Bootloader /home/codio/anaconda3/envs/day/lib/python3.11/site-packages/PyInstaller/bootloader/Linux-64bit-Intel/run
28996 INFO: checking EXE
28996 INFO: Building EXE because EXE-00.toc is non existent
28996 INFO: Building EXE from EXE-00.toc
28996 INFO: Copying bootloader EXE to /home/codio/workspace/day/dist/day
28997 INFO: Appending PKG archive to custom ELF section in EXE
29034 INFO: Building EXE from EXE-00.toc completed successfully.
(day) codio@morningorlando-misterjoseph:~/workspace/day$ ./dist/day

-----
Today Is:
-----
Tuesday
25 July
2023
14:35:17

(day) codio@morningorlando-misterjoseph:~/workspace/day$ ./dist/day yesterday

-----
Yesterday Was:
-----
Monday
24 July
2023

(day) codio@morningorlando-misterjoseph:~/workspace/day$ ./dist/day tomorrow

-----
Tomorrow Will Be:
-----
Wednesday
26 July
2023

(day) codio@morningorlando-misterjoseph:~/workspace/day$
```

Guide x

Collapse

One-File Approach

Creating the Spec File

Start the `day` virtual environment.

conda activate day

While the single-folder approach has advantages from a debugging point of view, it is not very user friendly. Let's create a single executable file with Pyinstaller. But, before we do, let's remove the work done on the previous page. Enter the command below to remove the `build` and `dist` directories.

rm -r dist build

▼ Did you notice?

The remove command uses the `-r` flag. You cannot use `rm` to remove a directory. You first need to remove all of its contents first. The `-r` flag means to remove recursively. So the terminal will go into a directory and remove any files. If there are subdirectories, the terminal will go into that directory and remove any contents. Use the `-r` flag if you want to remove a directory from the command line.

Next, we want to rename the spec file so we can compare this file when using the one-folder approach and the one-file approach. Use the move command to overwrite `day.spec` with `old-day.spec`.

mv day.spec old-day.spec

On the previous page, we talked about the importance of the spec file. Let's create the spec file separately and then use it to build the executable program as a single file. Replace `pyinstaller` with `pyi-makespec` to just make a spec file. Use the same flags as before to set the name and path. In addition, use the `--onefile` flag to generate a single executable file.

pyi-makespec frame.py --name day --paths="/home/codio/workspace/day" --onefile

Once this is done, list the contents of the directory. You should see the `day.spec` file, but the `build` and `dist` directories do not yet exist.

ls

Let's compare the two spec files with the `diff` utility.

diff -c old-day.spec day.spec

Output with `*** 24,36 ****` means lines 24 through 36 in the `old-day.spec` file (because it was passed first to the `diff` utility), while `--- 24,40 ----` means lines 24 through 40 in the `day.spec` file. Any line with a `-` or `+` symbol indicates changes to make the two files identical. A `-` means remove a line from the first file to make them identical. A `+` means add the line to the first file to make them identical. Spec files are quite different when using the one-file and one-folder approaches.

The `pyi-makespec` command is a useful tool in that it generates just a spec file. This file acts like a blueprint for generating your executable. You can then edit this manually to ensure the resulting executable is exactly what you need and want.

Generating the Executable

One you have a spec file, you can use the `pyinstaller` command to generate the executable. Instead of passing the name of the Python script to Pyinstaller pass the newly created spec file.

pyinstaller day.spec

If we list the contents of the current directory, we should now see the `build` and `dist` directories.

ls

If we look inside the `dist` directory, we will see something different. Instead of another directory named `day` that contains the executable, you will see just the executable `day`. **Note**, directories are colored blue while executables are colored green.

ls dist

Our executable is located in the `dist` directory. Use `./` to run our program and followed by the path to the executable. It should run as expected.

./dist/day

Try these variations:

- Run the program so that it outputs information about yesterday.

./dist/day yesterday

- Run the program so that it outputs information about tomorrow.

./dist/day tomorrow

Finally, deactivate the `day` virtual environment.

conda deactivate

Reading Question

Fill in the blanks below.

- One-file - This approach is best for the end user. They do not need to worry about having several required files before running the executable.
- One-folder - This approach is best for developers. They have the ability to check the presence/contents of required files when troubleshooting their executable.

The correct answers are:

- One-file - This approach is best for the end user. They do not need to worry about having several required files before running the executable.
- One-folder - This approach is best for developers. They have the ability to check the presence/contents of required files when troubleshooting their executable.

Check It!

Next